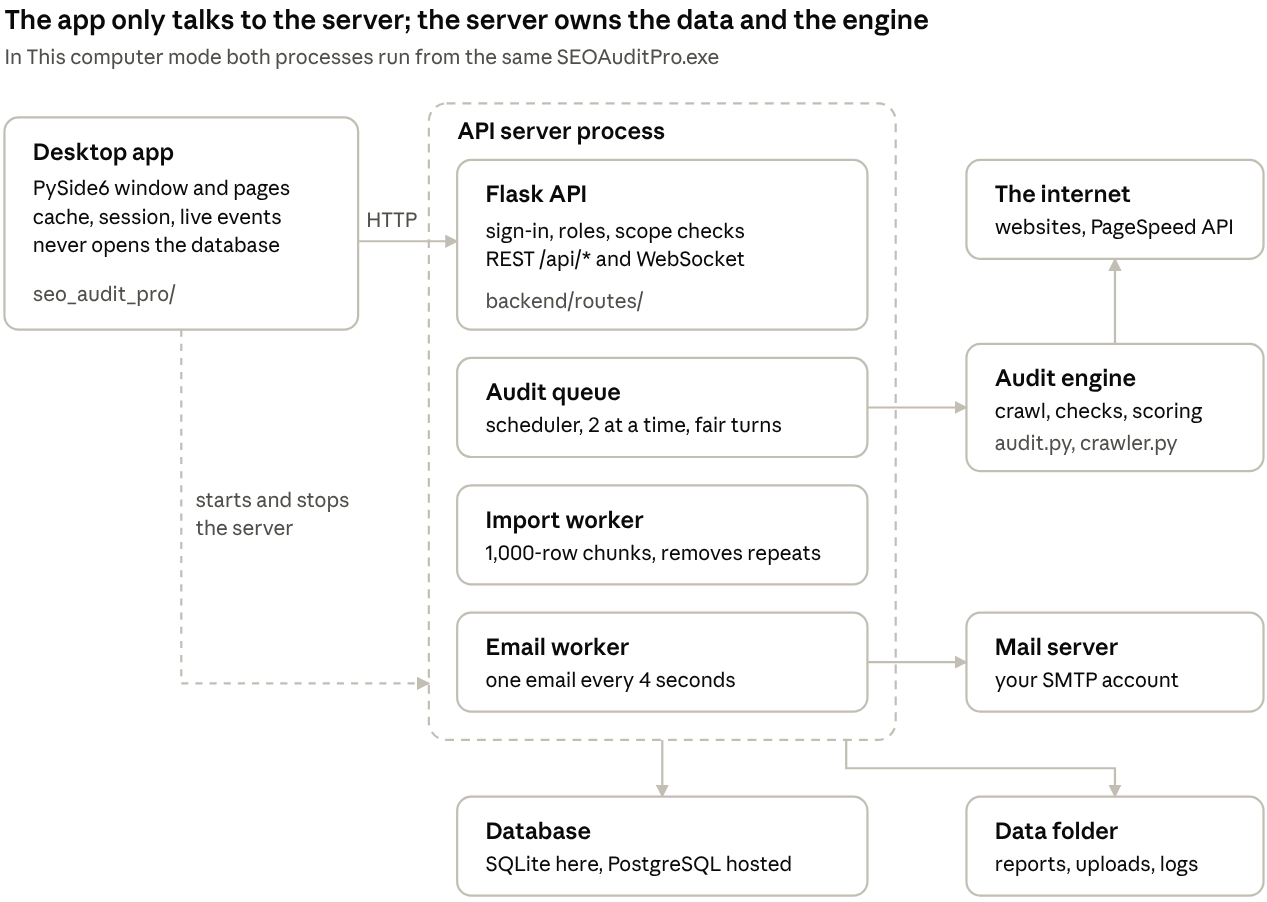
SEO Audit Pro — Technical Reference

Oct 21, 2026 · @Naweed Mirza

1. At a glance

SEO Audit Pro is two processes: a PySide6 desktop app and a Flask API server that owns the database, the files and the audit engine. The app reaches everything through the API, so the same app works with a server on the same computer or on a team server.



architecture · desktop app, server process, engine, storage

The app sends HTTP requests and receives live events over a WebSocket. Inside the server, three background workers run audits, imports and email; only they and the API touch the database, the data folder and the internet.

2. Project layout

One repository holds the desktop app, the API server and the audit engine. The engine files sit at the top level because they predate the rebuild and are reused as they are.

Path	What it is
`SEOAuditPro.pyw`	Entry point of the packaged program. Opens the app, or the server when started with `--serve`.
`seo_audit_pro/`	Desktop app (PySide6). Talks to the API only.
`seo_audit_pro/app.py`	Startup: splash, single instance, first-run flow, local server watch
`seo_audit_pro/local_server.py`	*On this computer* mode: starts, watches and stops the built-in server; also the server-side `serve()`
`seo_audit_pro/core/`	`api.py` (HTTP + token refresh), `session.py`, `tasks.py` (thread pool), `realtime.py` (WebSocket), `cache.py`, `settings.py`, `local.py`, `fmt.py`
`seo_audit_pro/ui/`	`auth.py` (welcome, setup, sign-in), `main_window.py`, `shell.py` (sidebar, top bar, navigation), `splash.py`
`seo_audit_pro/ui/pages/`	One module per screen, loaded on first use
`seo_audit_pro/widgets/`	Shared widgets: table, dialogs, toasts, skeletons, charts
`backend/`	Flask API server
`backend/routes/`	One blueprint per area: auth, org (users and teams), leads, audits, outreach, chat, workspace, admin
`backend/audit_runner.py`	Audit queue, scheduler and the run of one audit

Path	What it is
`backend/lead_import.py`	Background import of scraped files
`backend/results.py`	Compressed storage of audit results
`backend/reports.py`	Word reports, on-demand PDF, HTML report
`backend/migrations.py`	Versioned database schema
`audit.py`, `crawler.py`, `seo_checks.py`, `scoring.py`, `security_audit.py`, `pagespeed.py`, `structured_data.py`, `authority_traffic.py`	The audit engine
`report_docx.py`, `charts.py`, `templates/client_report.html`	Report generation
`web_app.py`	Server entry point for development and hosting
`tests/`	Automated tests
`scripts/`	Asset generation, development seed data, UI screenshots
`SEOAuditPro.spec`, `build_exe.bat`, `scripts/package_portable.py`	Building the exe and the portable zip
`docs/`	Markdown reference (architecture, API, database, security, deployment, testing)

3. Running from source

You need Windows, Python 3.12 or newer and Git. Everything else installs into a local virtual environment.

One-time setup

```
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt -r requirements-desktop.txt
```

`requirements.txt` is the server and engine; `requirements-desktop.txt` adds PySide6, keyring and PyInstaller.

Two ways to run it

A. Separate server and app (best for working on the API).

1. `run.bat` starts the API on `http://127.0.0.1:5055` with a SQLite database in `server_data\`.
2. Optional: `.venv\Scripts\python.exe scripts\dev_seed.py` creates a demo team and accounts. Their passwords are written to `server_data\dev_credentials.txt`.
3. `run_desktop.bat` opens the app. Run from source, it connects to `127.0.0.1:5055` by default.

B. Exactly like the packaged app (best for testing first run). Set a throwaway profile, then open the app:

```
set SEO_AUDIT_PRO_PROFILE=dev-first-run
.venv\Scripts\pythonw.exe SEOAuditPro.pyw
```

A profile gets its own settings, saved sign-in and data folder (`AppData\Local\SEO Audit Pro (dev-first-run)`), and starts at the Welcome screen like a fresh install. Delete that folder and the registry key `HKCU\Software\Mindcob\SEO Audit Pro (dev-first-run)` to start over.

Safety: your production database

If a `.env` file holds a `DATABASE_URL`, development runs **ignore it** and use SQLite. PostgreSQL is used only when `SEO_ENV=production`, or when you set `SEO_DB=postgres` on purpose. Tests always force SQLite in a temporary folder.

Useful commands

Command	What it does
`.venv\Scripts\python.exe -m unittest discover -s tests -t .`	Runs every automated test (about one minute)
`.venv\Scripts\python.exe -m pyflakes backend seo_audit_pro tests`	Lint

Command	What it does
`.venv\Scripts\python.exe -m backend.manage migrate`	Applies pending migrations
`.venv\Scripts\python.exe -m backend.manage create-manager --email you@x.com --name "You"`	Creates a Manager on a server (prints a temporary password)
`build_exe.bat`	Builds the exe and the portable zip (section 13)

4. Configuration

The server reads environment variables once at start (`backend/config.py`). A `.env` file next to the code, or next to the exe, is loaded too, but real environment variables win. The desktop app holds no secrets.

Environment variables

Variable	Default	What it does
`SEO_ENV`	`development`	`production` refuses to start without real secrets and enables PostgreSQL. The built-in server uses `local` .
`SEO_DB`	(empty)	`sqlite` forces SQLite; `postgres` allows PostgreSQL outside production
`DATABASE_URL`	(empty)	PostgreSQL connection string; ignored unless production or `SEO_DB=postgres`
`SEO_DATA_DIR`	`server_data/`	Database, uploads, reports, logs and generated keys

Variable	Default	What it does
`SEO_SQLITE_PATH`	`<data>/seo_audit_pro.db`	SQLite file location
`SECRET_KEY`	generated	Signs access tokens. Outside production it is created once and saved as `<data>/.secret_key`.
`DATA_ENCRYPTION_KEY`	generated	Encrypts stored secrets (SMTP password, PageSpeed key). Saved like `SECRET_KEY`.
`ACCESS_TOKEN_TTL`	1800	Access token lifetime in seconds (30 minutes)
`REFRESH_TOKEN_TTL`	1209600	Sign-in lifetime in seconds (14 days)
`LOGIN_ATTEMPTS_PER_IP_5MIN`	30	Sign-in attempts allowed per address per 5 minutes
`MAX_CONCURRENT_AUDITS`	2	Audits at the same time until a Manager changes it in Settings
`BULK_MAX_RECIPIENTS`	200	Leads per bulk email
`EMAIL_SEND_INTERVAL_SECONDS`	4	Pause between two outgoing emails
`DAILY_EMAIL_LIMIT`	500	Emails per day
`BOOTSTRAP_MANAGER_EMAIL`, `_PASSWORD`, `_NAME`	(empty)	Creates the first Manager on a hosted server when no account exists
`PAGESPEED_API_KEY`	(empty)	Google PageSpeed key, used when none is saved in Settings

Variable	Default	What it does
`GROQ_API_KEY`, `OPENROUTER_API_KEY`, `ANTHROPIC_API_KEY`, `GEMINI_API_KEY`	(empty)	Optional AI provider for the executive summary; without one a built-in summary is used
`MOZ_ACCESS_ID`, `MOZ_SECRET_KEY`	(empty)	Optional Moz data; without them authority is estimated
`PORT`, `HOST`	5055, 127.0.0.1	Where `web_app.py` listens

Settings stored in the database

Managers change these in the app; they live in `app_settings`.

Key	Holds
`audit_defaults`	JSON: `max_pages` (1–500), `psi_strategy` (`both` or `mobile`), `concurrent` (1–4)
`pagespeed_key`	The PageSpeed key, encrypted
(table `smtp_settings`)	Email server settings; the password is encrypted and never returned by the API

Desktop app settings

The app keeps only non-secret preferences in the Windows registry under `HKCU\Software\Mindcob\SEO Audit Pro`: `mode` (`local` or `remote`), `api_url`, `local_port`, `share_network`, window size and sidebar state. The refresh token for *Keep me signed in* is in Windows Credential Manager. `SEO_AUDIT_PRO_PROFILE` gives a separate set of all of these (section 3).

5. "This computer" mode

In this mode the app runs the full API server itself, as a second windowless process of the same exe. The app still talks to it over HTTP like any remote server, so the desktop code never touches the database. Code: `seo_audit_pro/local_server.py`,

`seo_audit_pro/core/local.py`, `seo_audit_pro/app.py`.

What happens at start

1. `SEOAuditPro.pyw` sees no `--serve` flag and opens the app. A `QLocalServer` named per Windows user makes sure only one window runs; a second launch tells the first to come forward and exits.
2. `settings.mode()` decides the path: no mode yet (packaged first run) shows **Welcome**; `remote` connects to the saved address; `local` starts the server.
3. `LocalServer.start()` picks a free port (5057 first, or the one used last time), then runs `SEOAuditPro.exe --serve --port P --host 127.0.0.1 --data <data folder> --parent <app pid>` with `CREATE_NO_WINDOW`.
4. The server process redirects its output to `<data>/logs/server-console.log`, sets `SEO_DB=sqlite`, `SEO_ENV=local` and an empty `DATABASE_URL`, then imports `web_app`. On a new data folder this creates the database (all migrations) and the secret keys.
5. The app polls `/api/health` (up to 120 s; about 3 s for the packaged build) and then `GET /api/setup/status`.
6. `needs_setup: true` shows **Create the Manager account**, which calls `POST /api/setup/manager`. Otherwise the normal sign-in or saved-session path runs.

Keeping it alive, and stopping it

Situation	What happens
App closes normally	`aboutToQuit` terminates the server process
App crashes or is killed	The server waits on the parent's process handle (`WaitForSingleObject`) and exits at once
Server crashes	A 5-second timer in the app notices and starts it again on the same port
Sharing turned on in Settings	The server restarts bound to `0.0.0.0`; the app shows `http://<LAN address>:<port>`

Audits that were running when the server stopped go back to the queue at the next start (once; a second interruption marks them failed).

First-run setup endpoints

- `GET /api/setup/status` → `{needs_setup, this_computer}`. Public.
- `POST /api/setup/manager` `{name, email, password}` → a normal sign-in payload (tokens and user).
- It works only while the `users` table is empty **and** the request comes from a loopback address. Otherwise it answers 403 (another computer) or 409 `already_set_up`. The password policy applies.

Data folder

`%LOCALAPPDATA%\SEO Audit Pro\data` holds `seo_audit_pro.db` (SQLite in WAL mode), `.secret_key`, `.data_encryption_key`, `reports/`, `uploads/`, `imports/` (files waiting to import), `logs/` and `server.json` (pid, port, host of the running server).

6. Database

The same schema runs on SQLite (local installs, development, tests) and PostgreSQL (hosted servers). All SQL uses `%s` placeholders; `backend/db.py` converts them for SQLite. Times are UTC ISO strings with milliseconds, so text order equals time order.

Tables

Area	Tables
People	`teams`, `users`, `sessions`, `password_resets`, `login_attempts`
Leads	`leads`, `lead_assignments` (history), `lead_batches` (one per imported file), `lead_batch_rejects` (each removed row and why)
Audits	`audits` (also the queue), `audit_batches` (bulk audits), `audit_results` (compressed engine output), `audit_issue_states` (resolved issues), `reports`
Outreach	`smtp_settings`, `emails`, `email_batches`, `email_attachments`, `uploads`

Area	Tables
Chat	`conversations`, `conversation_reads`, `messages`, `message_attachments`, `message_reactions`
System	`notifications`, `app_settings`, `audit_logs` (append-only), `idempotency_keys`, `schema_migrations`

Rules worth knowing:

- A website exists once: `leads.normalized_domain` is unique.
- A lead has at most one waiting or running audit (partial unique index).
- `leads.version` is checked on every edit; a stale save gets 409.
- Users are never deleted, only deactivated, so history never points at nothing.
- `audit_logs` rejects UPDATE and DELETE with a trigger on both databases.

Migrations

`backend/migrations.py` holds a numbered list. Each start applies the ones missing from `schema_migrations`, each in its own transaction.

Version	What it does
1	Initial schema
2	Lead bank: niche/area columns, Google Maps fields, import batches and rejects; drops the old CV bank
3	Audit queue: `audit_batches`, `audit_results`, queue columns on `audits`, `reports.ready`

On PostgreSQL, trigram search indexes are added after migrating if the `pg_trgm` extension is allowed; if not, search still works, only slower on huge banks.

Adding a migration

1. Write a function `_m004_<name>()` returning a list of SQL statements. Use `db.DIALECT` when the two databases need different SQL (see `_m003_audit_queue` for BLOB vs BYTEA).

2. Append `(4, "<name>", _m004_<name>)` to `MIGRATIONS`.
3. Never edit a migration that has shipped; add a new one.
4. Run the tests: they build a fresh database from all migrations.

7. Security model

Every rule is enforced by the server. The desktop app hides what a role cannot use, but that is only for comfort. Code: `backend/auth.py`, `backend/security.py`, `backend/routes/auth_routes.py`.

Sign-in and tokens

- Passwords are hashed with scrypt. Policy: 10+ characters, upper and lower case, a number, not a common password, not the email name.
- Sign-in returns a short **access token** (HMAC-signed, 30 minutes, tied to a session row) and a **refresh token** (random, stored only as a hash, 14 days).
- Each refresh **rotates** the refresh token; the old one stops working. The app refreshes once for all requests in flight.
- Five wrong passwords lock the email for 15 minutes. Sign-in is also limited per network address.
- Changing or resetting a password, or deactivating a user, revokes all their sessions and closes their live connections.
- New users get a temporary password and must change it before anything else works.

Who can see what

- `@require(MANAGER, TEAM_LEAD)` guards role-only endpoints; a denial is written to the audit log as `PERMISSION_DENIED`.
- `lead_scope(user)` adds the scope to every lead query: Managers see all, Team Leads their `assigned_team_id`, Agents their `assigned_agent_id`.
- Audits, reports, emails and activity are checked through their lead (`load_audit`, `load_lead`, `can_access_lead`).
- Anything outside a user's scope answers **404**, the same as something that does not exist, so ids cannot be probed.
- A duplicate website warning never reveals the id of a lead the user cannot see.

Other protections

Topic	How
Double clicks and retries	Create/send endpoints accept an `Idempotency-Key` header and replay the first answer
Concurrent edits	`leads.version` check; the loser gets 409
Stored secrets	SMTP password and PageSpeed key encrypted with Fernet, key derived from `DATA_ENCRYPTION_KEY`
SQL	Always parameterised; column names only from allow-lists; `LIKE` input escaped
Audit targets	Only public http(s) hosts on standard ports; private, loopback and link-local addresses refused before and during the run
Uploads	Type checked from the file's own bytes, size limits, safe stored names, served only to people allowed to see them
Email	Header injection refused; drafts come only from real audit results
Responses	`nosniff`, `X-Frame-Options: DENY`, no-referrer, `no-store` on JSON
Rate limits	Sign-in, audits, bulk audits, imports, exports, report regeneration
First-run setup	Only when no user exists and only from loopback
Activity log	Append-only table; UPDATE and DELETE refused by trigger

8. API overview

All endpoints are JSON over HTTP(S) under `/api`. The full list with every parameter is in `docs/api.md`.

Conventions

- Send `Authorization: Bearer <access token>` on everything except sign-in, refresh, password reset, health and first-run setup.
- Errors look like `{"error": "<message for people>", "code": "<machine code>", ...}` with the HTTP status. Common codes: `unauthorized` 401, `password_change_required` and `forbidden` 403, `not_found` 404, `conflict`, `possible_duplicate`, `count_changed`, `in_progress` 409, `invalid` 422, `rate_limited` and `locked` 429.
- Lists take `page` and `page_size` and return `{items, total, page, page_size, pages}`.
- Endpoints that create or send take an optional `Idempotency-Key` header.
- Bulk actions on "all matching" take a `filter` (the same keys as the lead list) and a count you confirm; if the count changed, the answer is 409 with the new numbers.

Endpoint groups

Group	Prefix	Main endpoints
Sign-in and profile	`/api/auth`	`login`, `refresh`, `logout`, `me`, `me/avatar`, `change-password`, `forgot-password`, `reset-password`
First run	`/api/setup`	`status`, `manager`
People	`/api/users`, `/api/teams`	list, create (returns a temporary password once), edit, deactivate, reactivate, reset password
Leads	`/api/leads`	list, `summary`, `facets` (niche → area tree), get, create, edit, assign, `bulk-assign` (single or split), `archive`, `restore`, `export` (CSV), activity
Imports	`/api/lead-imports`, `/api/lead-batches`	upload a file for preview, start, discard, list, progress, `rejects`, `rejects.csv`

Group	Prefix	Main endpoints
Audits	`/api/audits`	list, start, get (with queue position), cancel, `result`, issue state, `client-data`, `client-html`, `export`, `reports` (regenerate)
Bulk audits	`/api/audits/bulk`, `/api/audit-batches`, `/api/audit-queue`	preview, start, list, get, cancel, queue status
Reports	`/api/reports`	list, download (creates a PDF on first download)
Outreach	`/api/outreach`	SMTP settings and test, status, generate draft, emails, send, uploads, bulk preview and send
Chat	`/api/chat`	contacts, conversations, messages, read, edit, delete, reactions, uploads, attachments, search
Workspace	`/api/notifications`, `/api/dashboard`, `/api/search`	notifications and read, dashboard per role, global search
Admin	`/api/settings`, `/api/audit-logs`	audit defaults, PageSpeed key, activity log
Live events	`/api/ws`	WebSocket, see section 11
Health	`/api/health`	`{status, version, app}`

9. Lead import pipeline

Imports are built for many large files: a file is read as a stream, written in chunks, and never held in memory whole. 20,000 rows import in about 3 seconds on SQLite. Code: `backend/lead_import.py`, routes in `backend/routes/lead_routes.py`.

1. **Upload and preview** (`POST /api/lead-imports`). The file (CSV, TSV, TXT or XLSX, up to 200 MB) is saved under `<data>/imports/`. A `lead_batches` row is created as `pending` with the headers, a detected column mapping and a niche/area guess from the file name. Nothing is imported yet.

2. **Column detection.** Header names are normalised (case, spaces, camelCase) and matched against known names for company, website, phone, email, address, category, rating, reviews, Maps URL, city and more. Columns that match nothing are kept as JSON in `leads.extra`.
3. **Start** (`POST /api/lead-batches/<id>/start`). Requires a website column, a niche and an area. Optional team or agent; Team Leads can only target their own team.
4. **Background run.** A small pool of 2 workers processes batches. Rows are read 1,000 at a time:
   - no website → rejected `no_website`; a website that is only a Google Maps, Facebook or similar link, or not a real address → `invalid_website`
   - the domain or phone already seen earlier in the same file → `duplicate_in_file`
   - the domain or phone already in `leads` → `duplicate_existing` (the existing lead id is kept, but only shown to people who may see it)
   - the rest are inserted in one bulk statement; a race on the unique domain is retried row by row
5. **Progress** is saved after each chunk (`processed_rows`) and pushed live as `import_progress`. If the server stops, `resume_interrupted()` continues from the last saved row at the next start.
6. **Finish.** The lead caches are cleared, the batch is marked `completed`, an audit-log entry and a notification are written, and the uploaded file is deleted. Pending previews nobody started are cleaned up later.

Ratings with a decimal comma (`4,9`) are read correctly; ratings are kept between 0 and 5. Two-letter state codes keep their capitals in guessed areas (`Dallas TX`).

10. Audits: queue, engine, storage, reports

The queue is the `audits` table itself: rows with `status = 'queued'` are waiting, so the line survives a restart. Code: `backend/audit_runner.py` (queue and run), `audit.py` and friends (engine), `backend/results.py`, `backend/reports.py`.

Queue and scheduler

- `start()` (single audit) inserts a row with `priority = 1`; `enqueue_bulk()` inserts one row per lead with `priority = 0`, all in one `audit_batches` batch, using a bulk insert.
- One scheduler thread wakes on every enqueue and finish (and every 5 s). While fewer than `concurrency()` audits run (Settings, 1–4, default 2), it picks the next one:

    1. the oldest single audit, if any;
    2. otherwise **fair turns**: among people with waiting bulk audits, the one with the fewest running, then the one served longest ago; their oldest waiting audit goes next.
- Claiming is a conditional `UPDATE … WHERE status = 'queued'`, then a worker thread runs the audit.
- Cancelling a waiting audit just changes its row; a running one gets a cancel event the engine checks between steps.
- A finished batch writes one `audit_batch_completed` notification for its creator. Audits inside a batch send no per-lead notifications.
- Bulk limits live in `backend/routes/audit_routes.py`: `BATCH_CAP` (100, Managers 500) and `QUEUE_CAP` (300 waiting per person, Managers 2,000). Leads audited in the last 30 days are skipped unless the caller asks to include them.
- Time estimates use the average of the last 20 finished audits. A bulk batch is assumed to get `workers / people waiting` of the capacity.

Running one audit

1. The address is checked again (public host, standard port).
2. `audit.run_audit()` runs the engine with the saved settings: max pages, PageSpeed strategy, the PageSpeed key.
3. Engine stages, in order: fetch the homepage → robots.txt and sitemap → crawl (8+ parallel fetches) → analyse pages → internal and broken links → PageSpeed Insights (mobile and desktop) → structured data → SSL → passive security checks → authority and traffic estimates → scoring → executive summary.
4. Progress comes only from the engine's own messages (step text, pages read and found), saved at most once a second and pushed live as `audit_progress`.
5. The result is saved, reports are made, and the audit and lead rows are updated in one transaction.

Page fetching limits

`crawler.fetch_page()` streams every response. Binary files (images, PDFs, archives, Office files) are never downloaded; other non-HTML types are checked from their first 4 KB; HTML is capped at 3 MB and decoded with the charset the page declares (detection only on a 64 KB sample). This keeps a 50-page audit of a site with huge pages at about a minute and under 200 MB.

Result storage

`results.save()` writes the full engine output (about 0.5–1.5 MB of JSON) gzip-compressed into `audit_results` (about 50–80 KB). `results.load()` returns a fresh copy each time and keeps a small cache of the compressed bytes. Audits saved by older versions are read from the legacy `history_manager` store once and copied over.

Reports

- When an audit finishes, `reports.generate_all()` writes the client and technical **Word** files and adds a **PDF** row with `ready = 0`.
- The first download or email of that PDF calls `ensure_pdf()`, which converts the Word file with LibreOffice or Microsoft Word (PowerShell COM). One conversion runs at a time (a lock); a request waits up to 4 minutes, then gets 503 `busy`.
- The HTML client report is rendered on request from `templates/client_report.html`.
- *Regenerate reports* rewrites the Word files and resets the PDFs to be made again on next open.

11. Email, live updates and chat

Email queue

Code: `backend/mailer.py`, `backend/routes/outreach_routes.py`.

- Sending only queues an email (`status = 'queued'`). One background worker sends them one at a time, waiting `EMAIL_SEND_INTERVAL_SECONDS` between two, and stops for the day at `DAILY_EMAIL_LIMIT`.
- Each send connects to the saved SMTP server (STARTTLS, SSL or none), attaches reports and uploads, and marks the email `sent` or `failed` with the server's reason. A sent email sets the lead's `last_contact_at` and status *Contacted*.
- Drafts are generated only from the lead's latest completed audit (score and top real findings). There are no made-up numbers.
- Bulk send previews first: it skips leads without a valid email, without an audit, emailed in the last 14 days, or Won, Lost or Archived, then needs the exact count confirmed.
- The same worker also runs maintenance: it removes old idempotency keys, sign-in attempts, reset codes, expired sessions and abandoned import previews.

Live updates (WebSocket)

Code: `backend/realtime.py` (server), `seo_audit_pro/core/realtime.py` (app).

- The app connects to `/api/ws` and first sends `{"type": "auth", "token": "<access token>"}`; the server answers `ready`.
- The app pings every 25 seconds; the server checks the session on each ping and closes the socket if the user signed out or was deactivated.
- Events: `notification`, `message`, `message_updated`, `typing`, `presence`, `audit_progress`, `audit_batch`, `import_progress`. The publisher decides who receives each event, so nobody gets events about things they may not see.
- Each connection has its own outbox (500 frames) and sender thread. A client that stops reading never blocks an audit or a request; when its outbox is full the server closes it and the app reconnects.
- While the socket is down, the app reconnects with backoff, and the audit and chat screens poll instead.

Chat

Code: `backend/routes/chat_routes.py`, `backend/chat_access.py`.

- One team channel per team (Managers can open all of them) and one direct conversation per pair of people.
- `chat_access` decides who may message whom: Managers anyone; Team Leads and Agents the Managers and their own team.
- Messages support replies, edits and deletes by their author, four reactions, attachments and search within the conversations you belong to. Unread counts come from `conversation_reads`.

12. Desktop app internals

The app is PySide6 (Qt Widgets). Its one rule: the window thread never waits on the network or the disk.

Part	File	What it does
Background work	`core/tasks.py`	`tasks.run(fn, *args, on_done=, on_error=, owner=)` runs `fn` on a thread pool and calls back on the window thread. Callbacks are skipped if `owner` was closed in the meantime.
API client	`core/api.py`	One `requests` session. Adds the token, refreshes it once when it expires (other calls wait for that refresh), turns errors into `ApiError(status, message, code, data)`, and tracks Online/Offline. Downloads stream to disk.
Session	`core/session.py`	Tokens and the signed-in user. With *Keep me signed in*, the refresh token goes to Windows Credential Manager via `keyring`.
Cache	`core/cache.py`	Keeps recent GET answers. `Page.fetch()` shows a cached answer at once and refreshes it in the background (skipped if younger than 8 s). Any change you make, and any live event about data, clears it.
Live events	`core/realtime.py`	The WebSocket; passes each event to every open page's `on_realtime(ev)`
Settings	`core/settings.py`	Registry preferences, the mode, the address of the server
Local server	`local_server.py`, `core/local.py`	Section 5
Shell	`ui/shell.py`	Sidebar per role, top bar, search, notifications popover, navigation with history (Back restores a page with the same parameters)

Part	File	What it does
Pages	`ui/pages/*.py`	One class per screen, registered in `ui/pages/__init__.py` and imported on first use. `activate(**params)` is called each time a page is shown.
Widgets	`widgets/`	`DataTable` (model/view table with server sorting, paging and columns that hide on narrow screens), `StatefulArea` (loading, empty, error, offline states), `Skeleton`, dialogs, toasts
Theme	`theme.py`, `icons.py`	Colours, the stylesheet and the line icons

Adding a screen

1. Create `ui/pages/<name>.py` with a `Page` subclass; set `nav_key` and `title`.
2. Load data with `self.fetch("key", api.get, "/api/…", on_done=…, on_error=self.area.error)`; show results inside a `StatefulArea`.
3. Register it in `ui/pages/__init__.py` and, for the sidebar, in `navigation_for()` in `ui/shell.py`.
4. Add the module to `hiddenimports` in `SEOAuditPro.spec`, because pages are imported by name at run time.

The app writes a log to `%LOCALAPPDATA%\SEO Audit Pro\logs\client.log`. Unhandled errors are logged there and shown as a short message, never as a crash dialog.

13. Building and packaging

One command builds everything: `build_exe.bat`. It takes about 5 minutes and makes a 200 MB app folder and a portable zip of about 90 MB.

What `build_exe.bat` does

1. Creates `.venv` if missing and installs `requirements.txt` and `requirements-desktop.txt`.
2. Regenerates the brand assets and the multi-size `app.ico` (`scripts/build_assets.py`).

3. Runs PyInstaller with `SEOAuditPro.spec` → `dist\SEOAuditPro\SEOAuditPro.exe` plus its `_internal` folder.
4. Runs `scripts/package_portable.py` → `dist\SEO-Audit-Pro-Portable-<version>.zip`, with a *READ ME FIRST.txt* and the two PDF manuals from `docs\manuals\`.
5. If Inno Setup 6 is installed, also builds an installer from `installer\SEOAuditPro.iss`.

Inside the spec

- **One exe, two roles.** `SEOAuditPro.pyw` opens the app, or the server when started with `--serve`. Both the server and the engine are bundled.
- **One folder, not one file.** The app starts in about 2 seconds and antivirus tools flag one-folder builds less often. The exe must stay next to `_internal`.
- **Data files** the engine reads at run time are bundled on purpose: `templates/`, `static/libs/`, and the data of `trafilatura`, `justext` and `tld`. Package metadata is copied for libraries that read their own version.
- **Left out:** `psycopg2` (local installs use SQLite), unused Qt modules and plugins (Quick, QML, PDF, WebEngine, the software OpenGL renderer), and tkinter. matplotlib is limited to its Agg backend.
- **Never bundled:** `.env`, databases and generated keys. The spec filters them, and the packaging script refuses to zip a build that contains any.
- **Icon and version:** `seo_audit_pro/assets/app.ico` and `installer/version_info.txt`. Change the version in `seo_audit_pro/__init__.py`, `backend/config.py` and `installer/version_info.txt` together.

Checking a build

Tests cannot see packaging mistakes, so a new build should be run once:

1. Start `dist\SEOAuditPro\SEOAuditPro.exe --serve --port 5091 --data <empty folder>` and call `/api/health`. A missing module or DLL shows up in `<folder>\logs\server-console.log`.
2. Run a real audit through that server and download its Word and PDF reports.
3. Open the app with `SEO_AUDIT_PRO_PROFILE=check` set, so it starts like a fresh install, and go through Welcome and setup.

The release build was checked this way: server start 2.8 s, real audits, Word and PDF reports, CSV and Excel import, export and chat all passed with no errors in the logs. Closing the app, or killing it, also stopped its server.

Signing

The exe is not code-signed, so Windows SmartScreen warns on first launch. A code-signing certificate and `signtool sign` on the exe would remove that warning.

14. Running a team server

A team can share one workspace in two ways.

Option A: one office computer

The simplest way. One computer runs the app in *On this computer* mode, and the Manager turns on **Settings › This computer › Let other computers on this network use this workspace**. The server then listens on all network addresses on the same port. Others choose *Connect to a team server* and type the address shown. That computer must stay on with the app open, and Windows Firewall must allow the app on private networks. Traffic on the office network is plain HTTP.

Option B: a hosted server

For teams in different places. `render.yaml` and `Procfile` describe a ready setup:

- Start command: `gunicorn web_app:app --workers 1 --threads 32 --timeout 180`.
- **Exactly one process.** The live-event hub, the audit scheduler and the email worker live in memory, so a second process would split them. Use threads for concurrency.
- Set `SEO_ENV=production`, `DATABASE_URL` (PostgreSQL, for example Neon), `SECRET_KEY` and `DATA_ENCRYPTION_KEY` (long random values; keep them, or stored SMTP passwords become unreadable), and `SEO_DATA_DIR` on a persistent disk.
- The first start runs all migrations. Create the first Manager with `BOOTSTRAP_MANAGER_EMAIL` / `_PASSWORD` / `_NAME`, or `python -m backend.manage create-manager`.
- Serve it over HTTPS. Without a persistent disk, report files and uploads are lost on redeploy (results stay in the database; reports can be regenerated).
- PDF reports need LibreOffice (`soffice` on PATH) on Linux hosts.

- Users install the same portable app and choose *Connect to a team server* with the https address.

Audit results take about 80 KB per audit in the database, so 10,000 audits need under 1 GB. Size the server like a 4 GB machine for 2 audits at a time, 8 GB for 3 or 4.

15. Tests and quality checks

69 automated tests cover the server, the queue, imports, the engine's fetch limits, *This computer* mode and the app's core. They run against a throwaway SQLite database in a temporary folder and never touch a real database (`tests/helpers.py` forces this).

```
.venv\Scripts\python.exe -m unittest discover -s tests -t .
```

File	Tests	What it proves
`test_api.py`	34	Sign-in, lockout, token rotation, forced password change, reset codes; role rules and 404 for out-of-scope ids; duplicates and version conflicts; the full audit → report → email flow with a real stored engine result and a local SMTP server; chat permissions; notifications; dashboard
`test_lead_bank.py`	7	Column detection and niche/area guessing; removal of rows without a website and of duplicates (in the file and in the bank, by domain or phone); rejected-row reports; facets; select-all assign and split; export permissions; 20,000-row import speed
`test_audit_queue.py`	11	Bulk preview skips and caps; stale confirmation; scope; never more than the set number of audits at once; single audits first; fair turns; restart recovery; batch cancel; one notification per batch; compressed results; legacy results; PDFs made on first use, one at a time
`test_engine_fetch.py`	6	Huge pages truncated; binary files not downloaded; HTML sent as text/plain detected; declared charset used; a stalled live client never blocks a publisher

File	Tests	What it proves
`test_local_mode.py`	5	First Manager created once, only locally, with the password policy; PageSpeed key saved encrypted; the built-in server starts with its own data folder and exits when its parent dies
`test_client_core.py`	6	The app's API client against a live test server: silent token refresh, expired sessions, friendly 403/404 messages, idempotent replays, an offline server, streamed downloads

Audits in the tests replay a real stored engine result (`tests/fixtures/real_audit_cancomco.json`) instead of crawling the internet, so they are fast and repeatable. Real crawls are checked by hand on a build (section 13).

`scripts/ui_snapshot.py` renders screens of the app against a running server and saves pictures, to check layouts at 1440×900 and 1366×768.

16. Extending the app, pitfalls and limits

Adding an endpoint

1. Add the route to the right blueprint in `backend/routes/`. Protect it with `@require(...)` and, for lead data, filter with `lead_scope()` or load with `load_lead()` / `load_audit()`.
2. Validate input with the helpers in `backend/util.py` (`text`, `email_value`, `page_args`) and raise `ApiError(status, message, field=...)` with a message a person can read.
3. Writes: one `db.transaction()`, an `events.audit(...)` entry, then `cache.bump()` after the commit if lead counts can change. Add `@idempotent` to anything that creates or sends.
4. Add a test, including one for a user who must **not** be allowed.

Common pitfalls

Pitfall	Do this instead
Network or disk work on the window thread	`tasks.run(...)` and update the screen in `on_done`
A new page module missing in the packaged exe	Add it to `hiddenimports` in the spec
An engine library that reads data files	Add `collect_data_files("<package>")` to the spec and test the build
Returning 403 for another user's record	Return 404 through the load helpers, so ids cannot be probed
More than one server process	Keep `--workers 1`; scale with threads
Running from source with a production `DATABASE_URL` in `.env`	It is ignored unless `SEO_DB=postgres`; keep it that way for development
Changing a shipped migration	Add a new migration
Changing `DATA_ENCRYPTION_KEY`	Stored SMTP passwords and the PageSpeed key must then be entered again

Known limitations

- Windows only for the desktop app.
- In *This computer* mode, audits run and others can connect only while the app is open.
- Office sharing is plain HTTP on the local network; use a hosted server with HTTPS across the internet.
- The exe is not code-signed (SmartScreen warns once).
- PDF reports need Word or LibreOffice where the server runs.
- PageSpeed data needs a Google API key for reliable results; authority and traffic are estimates unless Moz keys are set.
- The crawler follows redirects; a public site that redirects to an internal address is checked at the start only.
- Email opens and clicks are not tracked.
- One server process by design; very large teams would need a shared message broker to run several.